Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie

AGH University of Krakow

UAV Trajectory Planning

Hubert Szolc[1]

szolc@agh.edu.pl

[1]Embbeded Vision Systems Group
Computer Vision Laboratory
Deaprtment of Automatic Control and Robotics
EAIiIB Faculty
AGH University of Krakow, Poland

Outline

1. What is 'Trajectory'?
2. Good UAV model property
3. Different UAV trajectories

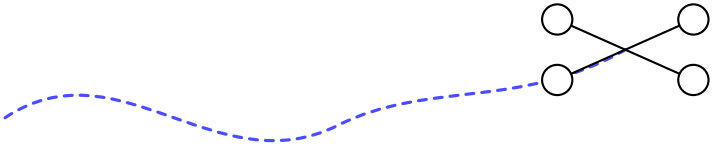
www.agh.edu.pl

Part I

Motivation

www.agh.edu.pl

Trajectory – intuition



Intuitively – just a line that is drawn by a flying drone.

www.agh.edu.pl

Trajectory – formally

$$x = \begin{bmatrix} p(t) & v(t) & q(t) & \omega(t) \end{bmatrix}^T$$

Formally, the trajectory is the **state** of the UAV over time.

www.agh.edu.pl

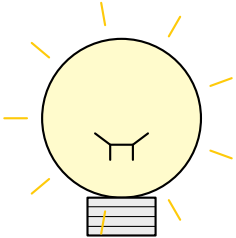
Trajectory – theoretically

$$\begin{bmatrix} \dot{p} \\ \dot{v} \\ \dot{q} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} v \\ -g + \frac{1}{m} q \odot T \\ q \otimes \left(\frac{\omega_x}{2}\mathbf{i} + \frac{\omega_y}{2}\mathbf{j} + \frac{\omega_z}{2}\mathbf{k}\right) \\ J^{-1}(\tau - \omega \times J\omega) \end{bmatrix}$$

From theoretical point of view, trajectory is a **solution** of the state equation.

www.agh.edu.pl

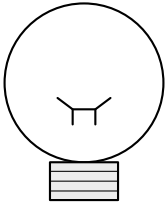
Clever solution?



If trajectory is a solution of the state equation, then we can just solve it for specific initial conditions and we will have a very good plan for the trajectory, right?

www.agh.edu.pl

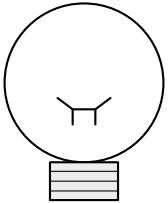
Clever solution? Well, not



- It's just a **model** – *sim2real* gap is a ~~bitch~~ big issue.

www.agh.edu.pl

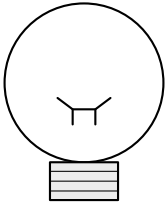
Clever solution? Well, not



- It's just a **model** – *sim2real* gap is a ~~bitch~~ big issue.
- It is a **nonlinear** model – no analytical solution.

www.agh.edu.pl

Clever solution? Well, not

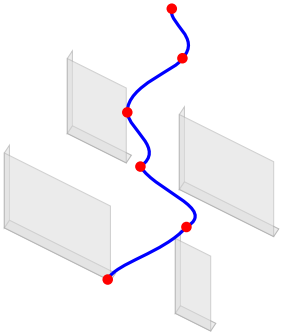


- It's just a **model** – *sim2real* gap is a ~~bitch~~ big issue.
- It is a **nonlinear** model – no analytical solution.
- This model has **inputs** – they need to be known somehow.

www.agh.edu.pl

Typical solution

1. Determine waypoints & obstacles.
2. Define optimization criterion, like minimize time.
3. Solve the optimization problem.



www.agh.edu.pl

Part II

Differential flatness

www.agh.edu.pl

Formal definition

Flatness [M. Fliess et al. 1992]

A nonlinear system:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}), \quad \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{u} \in \mathbb{R}^m$$

is flat, if there exists an output:

$$\mathbf{y} = (y_1, ..., y_m)$$

that satisfies the following conditions:

1. $\mathbf{y} = \Phi(\mathbf{x}, \mathbf{u}, \dot{\mathbf{u}}, \cdots, \mathbf{u}^{(\alpha)})$
2. $\mathbf{x} = g(\mathbf{y}, \dot{\mathbf{y}}, \cdots, \mathbf{y}^{(\beta)}), \quad \mathbf{u} = h(\mathbf{y}, \dot{\mathbf{y}}, \cdots, \mathbf{y}^{(\beta)})$
3. $\psi(\mathbf{y}, \dot{\mathbf{y}}, \cdots, \mathbf{y}^{(\beta)}) = \mathbf{0}$ only if ψ is a null function (differential independence)

www.agh.edu.pl

Building intuition

Linear (LTI) systems
$\dot{x} = Ax + Bu$
Controllability (Kalman):
$\operatorname{rank}\left[B \;\; AB \;\; \cdots \;\; A^{n-1}B\right] = n$

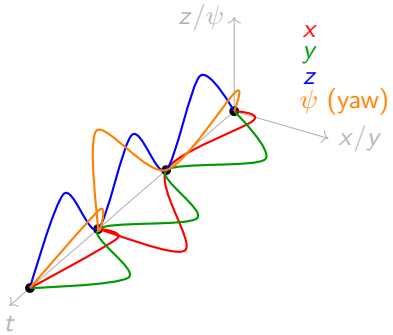
generalization
(LTI: controllable $\Leftrightarrow$ flat)

Nonlinear systems
$\dot{x} = f(x, u)$
Differential flatness:
there exists an output y such that
$x = g(y, \dot{y}, \dots), \quad u = h(y, \dot{y}, \dots)$

www.agh.edu.pl

UAV flat outputs

UAV model has 4 inputs $(T, \tau_x, \tau_y, \tau_z)$, hence we are looking for 4 flat outputs. One of the possible solution is to use the **position** with **yaw** angle.



www.agh.edu.pl

UAV flat outputs – proof [Mellinger & Kumar 2011]

If

$$\sigma = \begin{bmatrix} x & y & z & \psi \end{bmatrix}$$

are *true* flat outputs, then we should be able to obtain from them (and their derivatives) the entire state vector:

$$x = \begin{bmatrix} p & v & q/R & \omega \end{bmatrix}^T$$

as well as control inputs:

$$u = \begin{bmatrix} T & \tau \end{bmatrix}^T$$

www.agh.edu.pl

UAV flat outputs – position & velocity

No problem with that since:

$$\mathbf{p} = \begin{bmatrix} x & y & z \end{bmatrix}$$

$$\mathbf{v} = \begin{bmatrix} \dot{x} & \dot{y} & \dot{z} \end{bmatrix}$$

www.agh.edu.pl

UAV flat outputs – total thrust

Based on the model second equation:

$$m\mathbf{a} = -mg\mathbf{z_W} + T\mathbf{z_B}$$

$$T\mathbf{z_B} = m\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} + g \end{bmatrix}$$

$$T\mathbf{z_B} = m\mathbf{a_g}$$

So the value of thrust is equal to

$$T = m\|\mathbf{a_g}\|$$

www.agh.edu.pl

UAV flat outputs – rotation

We can express the $\mathbf{z_B}$ in the world frame:

$$T\mathbf{z_B} = m\mathbf{a_g}$$
$$\mathbf{z_B} = \frac{\mathbf{a_g}}{\|\mathbf{a_g}\|}$$

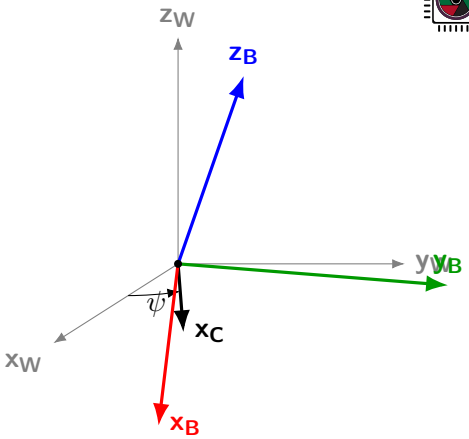
Then we determine the desired heading:

$$\mathbf{x_C} = \begin{bmatrix}\cos\psi & \sin\psi & 0\end{bmatrix}^T$$

and based on that we place the body frame:

$$\mathbf{y_B} = \frac{\mathbf{z_B} \times \mathbf{x_C}}{\|\mathbf{z_B} \times \mathbf{x_C}\|}$$
$$\mathbf{x_B} = \mathbf{y_B} \times \mathbf{z_B}$$

which gives us: $\mathbf{R} = \begin{bmatrix}\mathbf{x_B} & \mathbf{y_B} & \mathbf{z_B}\end{bmatrix}$



www.agh.edu.pl

UAV flat outputs – angular velocity (1)

Based – again – on the model second equation:

$$m\mathbf{a} = -mg\mathbf{z_W} + T\mathbf{z_B} \quad / \frac{d}{dt}$$

$$m\dot{\mathbf{a}} = \dot{T}\mathbf{z_B} + T\dot{\mathbf{z}}_\mathbf{B}$$

$$= \dot{T}\mathbf{z_B} + T(\omega \times \mathbf{z_B})$$

Now, by projecting it along $\mathbf{z_B}$:

$$m\dot{\mathbf{a}}\mathbf{z_B} = \dot{T}$$

By substitution to the previous equation:

$$m\dot{\mathbf{a}} = m(\dot{\mathbf{a}}\mathbf{z_B})\mathbf{z_B} + T(\omega \times \mathbf{z_B})$$

$$(\omega \times \mathbf{z_B}) = \frac{m}{T}(\dot{\mathbf{a}} - (\dot{\mathbf{a}}\mathbf{z_B})\mathbf{z_B}) = \mathbf{h}_\omega$$

Vector $\mathbf{h}_\omega$ can be viewed as the projection of the vector $\frac{m}{T}\dot{\mathbf{a}}$ onto the plane $\mathbf{x_B} - \mathbf{y_B}$.

www.agh.edu.pl

UAV flat outputs – angular velocity (2)

By cross product we have:

$$\mathbf{h}_\omega = \omega_y \mathbf{x_B} - \omega_x \mathbf{y_B}$$

Therefore:

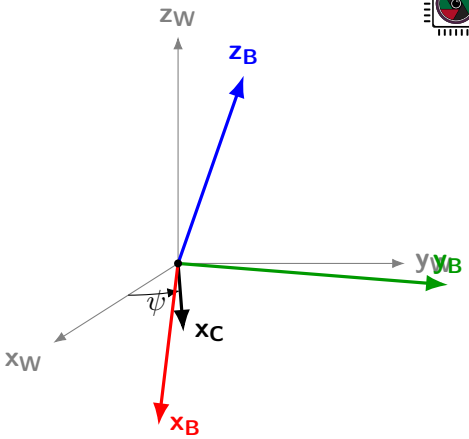
$$\omega_x = -\mathbf{h}_\omega \mathbf{y_B}$$
$$\omega_y = \mathbf{h}_\omega \mathbf{x_B}$$

To find ω_z we can simply note that:

$$\omega = \omega_{BC} + \omega_{CW}$$

and project it along $\mathbf{z_B}$:

$$\omega_\mathbf{z} = \omega_\mathbf{CW} \mathbf{z_B} = \dot{\psi} \mathbf{z_W} \mathbf{z_B}$$



UAV flat outputs – torque

To obtain torque we can use model last equation:

$$\dot{\omega} = J^{-1}(\tau - \omega \times J\omega)$$
$$\tau = J\dot{\omega} + \omega \times J\omega$$

So, we *only* need to find the angular acceleration $\dot{\omega}$. To do that we can use the same approach as with angular velocity, but take the second derivative of the force equation.

After that and *few more* calculation we will end up with the following formulas:

$$\dot{\omega}_x = \frac{m}{T}\ddot{a}_x - 2\frac{m}{T}\dot{a}_z\omega_y - \omega_x\omega_z$$
$$\dot{\omega}_y = \frac{m}{T}\ddot{a}_y + 2\frac{m}{T}\dot{a}_z\omega_x - \omega_y\omega_z$$
$$\dot{\omega}_z = \ddot{\psi}\mathbf{z_W}\mathbf{z_B}$$

www.agh.edu.pl

UAV flatness

Key takeaway

Thanks to the UAV model flatness, when planning trajectory it is enough to find only 4 differentiable functions: $x(t)$, $y(t)$, $z(t)$ and $\psi(t)$.

www.agh.edu.pl

Part III

Different ways of planning trajectory

www.agh.edu.pl

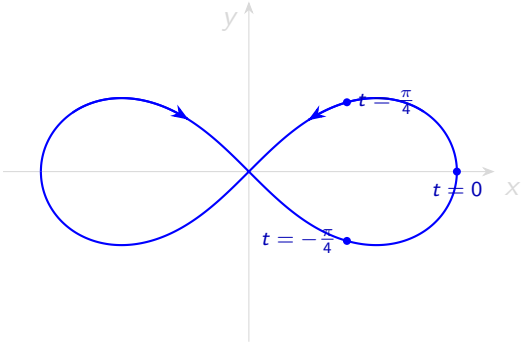
Predefined shapes

The simplest approach: just **parametrize** some curve and treat the parameter as a time.

Example: **lemniscate of Bernoulli**

$$x(t) = a\,\frac{\cos t}{1+\sin^2 t},$$
$$y(t) = a\,\frac{\sin t \cos t}{1+\sin^2 t}, \qquad t \in [-\pi, \pi].$$

- t is the curve parameter (often treated as time)
- arrows show the direction of increasing t



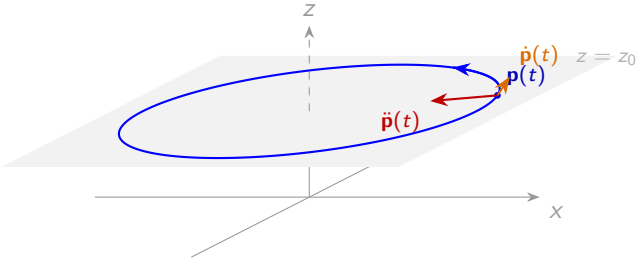
www.agh.edu.pl

Case study: circular trajectory

A standard circle in the plane $z = z_0$ can be parametrized as

$$
\begin{aligned}
x(t) &= R\cos(\omega t), & \dot{x}(t) &= -R\omega\sin(\omega t), & \ddot{x}(t) &= -R\omega^2\cos(\omega t),\\
y(t) &= R\sin(\omega t), & \dot{y}(t) &= R\omega\cos(\omega t), & \ddot{y}(t) &= -R\omega^2\sin(\omega t),\\
z(t) &= z_0, & \dot{z}(t) &= 0, & \ddot{z}(t) &= 0.
\end{aligned}
$$

- R – radius, ω – angular speed, t – time
- speed magnitude is constant: $\|\dot{\mathbf{p}}(t)\| = R\omega$
- acceleration is centripetal: $\ddot{\mathbf{p}}(t) = -\omega^2\,\mathbf{p}_{xy}(t)$



www.agh.edu.pl

Circular trajectory – pros & cons

Pros:

- easy to find analytical derivatives up to any order,
- easy to limit maximum speed, acceleration, ...

Cons:

- not ideal as a base for spline,
- not optimal (even not sub-optimal) in any practical sense.

Good for **testing** and only for that.

Some possible modifications to make it more *interesting*:

- yaw angle control: $\psi(t) = \pi + \omega t$ (heading towards circle centre),
- vertical movement: $z(t) = kt$ (spiral trajectory).

www.agh.edu.pl

Polynomial trajectories

There are not many functions better than **polynomials**:

- they are continuos,

www.agh.edu.pl

Polynomial trajectories

There are not many functions better than **polynomials**:

- they are continuos,
- they are easily differentiable,

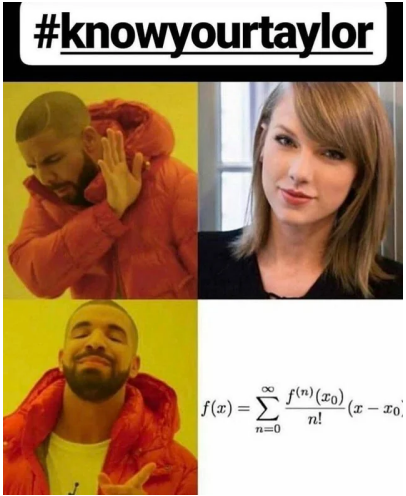
www.agh.edu.pl

Polynomial trajectories

There are not many functions better than **polynomials**:

- they are continuos,
- they are easily differentiable,
- they can approximate other functions ☺.

So let's use polynomials for trajectory planning!



Case study: minimum snap trajectories [Mellinger & Kumar 2011]

Find trajectory:

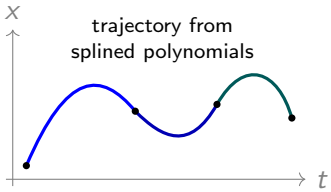
$$\sigma(t) = \begin{cases} \sum_{i=0}^{n} c_{0i}(t - T_0)^i & T_0 \leq t \leq T_1 \\ \sum_{i=0}^{n} c_{1i}(t - T_1)^i & T_1 \leq t \leq T_2 \\ \vdots & \\ \sum_{i=0}^{n} c_{(m-1)i}(t - T_{m-1})^i & T_{m-1} \leq t \leq T_m \end{cases}$$

that minimizes:

$$J = \int_0^{T_m} \left(\frac{d^4\sigma(t)}{dt^4}\right)^2 dt$$

with predefined time knots:

$$\sigma(t_j) = \sigma_j \quad \text{for} \quad j = 0, \cdots, m$$



Minimum snap solution

In order to make the problem easier to solve, we can use a polynomial algebra (ring) with time vectors as a base:

$$\sigma_j(t) = \begin{bmatrix} 1 & (t - T_j) & \cdots & (t - T_j)^7 \end{bmatrix} \begin{bmatrix} c_{j0} \\ c_{j1} \\ \vdots \\ c_{j7} \end{bmatrix}$$

With that we are able to find nice formula for k-th order derivative:

$${}^k\mathbf{t}[i] = \begin{cases} 0 & i < k \\ \frac{i!}{(i-k)!}(t - T_j)^{(i-k)} & i \geq k \end{cases}$$

Now our objective function transforms into:

$$\begin{aligned} J &= \sum_{j=0}^{m-1} \int_{T_j}^{T_{j+1}} ({}^4\mathbf{t}_j^T c_j)^T ({}^4\mathbf{t}_j^T c_j) dt \\ &= \sum_{j=0}^{m-1} \int_{T_j}^{T_{j+1}} c_j^T {}^4\mathbf{t}_j {}^4\mathbf{t}_j^T c_j dt \\ &= \sum_{j=0}^{m-1} c_j^T \left(\int_{T_j}^{T_{j+1}} {}^4\mathbf{t}_j {}^4\mathbf{t}_j^T dt \right) c_j \end{aligned}$$

www.agh.edu.pl

Minimum snap QP

Since

$$\int_{T_j}^{T_{j+1}} {}^4\mathbf{t}_j {}^4\mathbf{t}_j^T \, dt = M$$

is a constant matrix over time interval, we end my with a *well-known* Quadratic Programming (QP) formulation:

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2} c^T H c \\ \text{subject to} \quad & c_{lb} \leq c \leq c_{ub} \\ & a_{lb} \leq Ac \leq a_{ub} \end{aligned}$$

Further modifications:

- non-dimensional time,
- non-dimensional spatial dimension,
- time-knots optimization.

www.agh.edu.pl